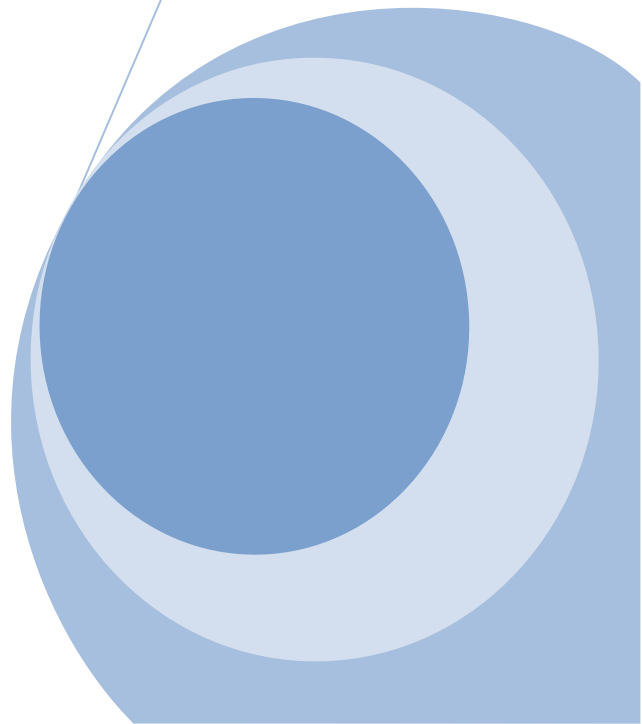




PART-3

OOPS

BY
Er. KAMLESH

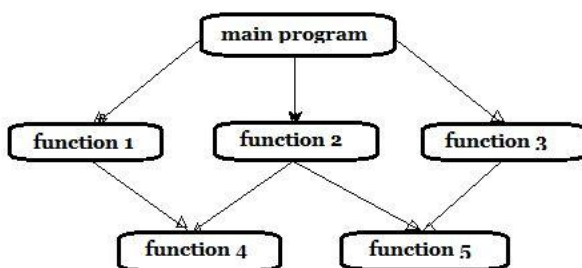


PROGRAMMING MODAL

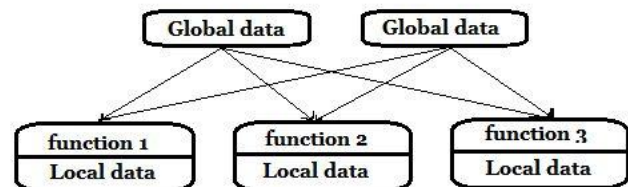
- ❖ Programming modal is use for develop any application.
- ❖ The basic purpose of any programming model to make programming efficient
Example: to make the process of writing a complex program less harder, bug free, easily understandable, easily modifiable etc.
- ❖ Two main programming modals available to develop application.
 - ⇒ Procedure oriented programming (POP)
 - ⇒ Object oriented Programming (OOP)

Procedure oriented programming (POP)

- ❖ COBOL, FORTRAN and C, is commonly known as procedure oriented programming (POP).
- ❖ Functions are the main building block of POP i.e. your application will be divided into multiple functions.



Structure of POP



Relationship of data and functions in POP

- ❖ In POP, Structure can be use represent the data.
Example: -

<pre> struct stack { int top; char * name; } Push() Pop() </pre>	<pre> struct queue { int front, rear; char * name; } insert() delete() </pre>
--	---

All functions are global

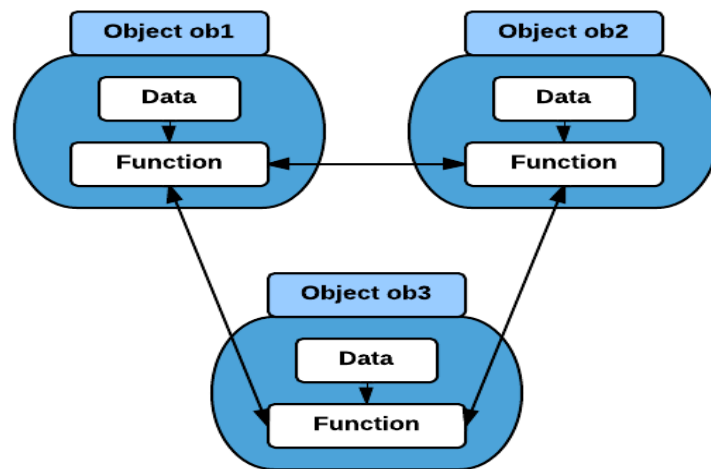
- ❖ Here we define one structure called stack with top and name and another structure called queue with front, rear and name.
- ❖ Two function are defined called push() and pop() which will operate on stack and two functions are defined called insert() and delete () which will operate on queue.
- ❖ In POP Languages, all the functions will be defined be defined globally i.e. there is no way to specify.
 - ⇒ Push (), pop () must be used with stack only.
 - ⇒ insert (), delete () must be used with queue only.

There is a chance that

- Push () can use queue.
 - Insert () can use stack.
- ⇒ This may cause problems like:
- ⇒ There is no owner for the data so it is very difficult to maintain data integrity.
- ⇒ Any function can use any data as shown above.
- ⇒ It is difficult to identify the function which is wrongly using the data etc.

Object oriented Programming (OOP)

- ❖ Object are the main building block of OPPS i.e. our applications will be divided into multiple objects.
- ❖ Father of OOPS is **Grady Booch**.



Data and functions in oops

Example:-

<pre> class stack { int top; String name[]; Push() Pop() } </pre>	<pre> class queue { int front, rear; String name[]; insert() delete() } </pre>
---	--

Here, we defined class called Stack with

- Data members – top and name.
- Methods - push () and pop () which will operate on data members of Stack.

We defined another class called Queue with

- Data member –front, rear and name.
- Methods-insert() and delete () which will operate

In OPP Languages, all the data and methods will be encapsulated in class.

- Push (), pop () are methods of Stack and can access data members of stack class.
- Insert (), delete () are methods of Queue and can access data members of Queue class.

Procedure oriented programming (POP)	Object oriented Programming (OOP)
Program is divided into small parts called functions .	Program is divided into parts called objects .
POP follows Top down approach .	OOP follows Bottom up approach .
POP does not have any access specifier	OOP has access specifier named Public, Private, Protected, etc.
Data can move freely from function to function in the system.	Objects can move and communicate with each other through member functions.
To add new data and function in POP is not so easy.	OOP provides an easy way to add new data and function.
POP does not have any proper way for hiding data so it is less secure .	OOP provides Data Hiding so provides more security .
In POP, Overloading is not possible.	Overloading is possible in the form of Function Overloading and Operator Overloading.
Example: C, VB, FORTRAN, Pascal.	Example: C++, JAVA, VB.NET, C#.

Object oriented Programming (OOPs) concepts:	Principles of OOPs
1. Object 2. Class 3. Abstraction 4. Encapsulation 5. Inheritance 6. Polymorphism 7. Dynamic binding 8. Message Communication	1. Abstraction 2. Encapsulation 3. Inheritance 4. Polymorphism

1. Object: -

- ❖ Everything in the world is an object.
- ❖ Object is physical representation of class.
- ❖ Object are the main building block of OPPS i.e. our applications will be divided into multiple objects.
- ❖ Grady Booch: -An Object has state, exhibits some well-defined behaviour and has a unique identity.
- ❖ An object has state, behaviour, and identity; the structure and behaviour of similar objects are defined in their common class; the terms instance and object are interchangeable.

2. Class: -

- ❖ Class is the Template, Blue print, grouping of state and behaviours.
- ❖ Class is a prototype or template or pattern of object's state and Behaviour.

3. Abstraction: -

- ❖ Collection of Information (It is on design level).
- ❖ Abstraction is a process of providing necessary properties and operation of an object.
- ❖ When you are describing object then you need to consider various perspective.
- ❖ When we are describing a Mobile for Customer then we need to provide only necessary properties and operation which are required for Customer by hiding some of the properties and operations which is required for Engineering.

4. Encapsulation: -

- ❖ Implementation of Abstraction is Encapsulation.
- ❖ Encapsulation is a process of describing the state and behaviour of an object in a class.

5. Inheritance: -

- ❖ Inheritance is the process of writing new class by inheriting commonly used state and behaviour of existing class.
- ❖ Existing class is called as **Super class or Base class or Parent class**.
- ❖ Newly defined class is called as **Sub class or Derived class or Child class**.

6. Polymorphism: -

- ❖ Polymorphism means (one name - Many forms)
- ❖ Polymorphism is the ability of an object to behave differently at different situation.

Example:-

1. Power button has polymorphic behaviour.
 - ⇒ When you press once, it will switch on the device.
 - ⇒ When you press it again device will be switched off.
2. +Operator in java has polymorphic behaviour.
 - ⇒ When operands are numeric then it will perform arithmetic addition.
 - ⇒ When the operands are of String type then it will perform String Concatenation.

7. Dynamic binding: -

- ❖ Dynamic binding means that the body associated with a given procedure call is not known until the time of the call at runtime.
- ❖ It associated with polymorphism and inheritance.
- ❖ Binding refers to the linking of a procedure call.

8. Message Communication: -

- ❖ The set of objects that communication with each other.
 - ⇒ Creating classes that define objects and their behaviour.
 - ⇒ Creating objects from class definition.
 - ⇒ Establishing communication among objects.